

100 Logical Reasoning Questions in C Programming

Bit Operations

Q1. `int a = 5, b = 3;`
`printf("%d", a & b);`

Q2. `int x = 6;`
`int result = x | 3;`
`printf("%d", result);`

Q3. `int x = 4;`
`printf("%d", x ^ x);`

Q4. `int x = 1;`
`x = x << 2;`
`printf("%d", x);`

Q5. `int x = 8;`
`x = x >> 1;`
`printf("%d", x);`

Q6. `~(5)` equals to?

Q7. Check if the 3rd bit in ``x = 10`` is set.

Q8. Swap two numbers using XOR.

Q9. What does `x & ~(1 << n)` do?

Q10. How to set the nth bit of a number?

Logical Operations

Q11. `int a = 0, b = 5;`
`printf("%d", a && b);`

Q12. `int a = 1, b = 0;`
`printf("%d", a || b);`

Q13. `!0` equals to?

Q14. `int x = 0;`
`printf("%d", x || ++x);`

Q15. Difference between ``&`` and ``&&`` in C?

Q16. What is the output of ``0 && 1 || 1``?

100 Logical Reasoning Questions in C Programming

Q17. Evaluate: `!(a && b) == !a || !b`

Q18. What is precedence: `&&` or `||`?

```
Q19. int a = 1;
int b = 1;
if (a-- && ++b)
printf("%d", b);
```

Q20. Can short-circuit evaluation prevent segfault?

1's and 2's Complement

Q21. What is 1's complement of 0101?

Q22. What is 2's complement of 0101?

Q23. Find -5 in binary using 2's complement.

Q24. Is `~x + 1` equal to `-x`?

Q25. Convert -10 to 2's complement in 8-bit.

Q26. Difference between signed and unsigned representation.

Q27. Overflow detection in 2's complement addition.

Q28. How is subtraction done using 2's complement?

Q29. What is the result of `~(-1)`?

Q30. 1's complement of -8 in 8-bit system?

Array

```
Q31. int arr[] = {1,2,3};
printf("%d", arr[1]);
```

Q32. Accessing out of bounds: `arr[10] = 5;`

Q33. Difference between `arr` and `&arr`.

Q34. Size of array using `sizeof` operator.

```
Q35. int a[5] = {0};
what is a[4]?
```

100 Logical Reasoning Questions in C Programming

Q36. How to pass array to function?

Q37. Can you modify an array in function?

Q38. What does `*(arr + i)` mean?

Q39. `int a[2][2] = {{1,2},{3,4}};`
`printf("%d", a[1][0]);`

Q40. Array of pointers vs pointer to array?

Linked List

Q41. What is a linked list?

Q42. How to traverse a linked list?

Q43. Detect loop in linked list?

Q44. Insert node at end in singly list.

Q45. Delete node in singly linked list.

Q46. Reverse a singly linked list.

Q47. Difference: singly vs doubly list.

Q48. Node definition with typedef.

Q49. Free all nodes in list.

Q50. Why use linked list over array?

Queue

Q51. What is FIFO?

Q52. Array-based implementation.

Q53. What causes queue overflow?

Q54. Enqueue and dequeue functions.

Q55. Circular queue logic?

Q56. Detect full in circular queue.

100 Logical Reasoning Questions in C Programming

Q57. Difference: queue vs stack.

Q58. Use queue to reverse string?

Q59. Queue using linked list?

Q60. Complexity of enqueue?

Function Pointers and Callbacks

Q61. Declare pointer to function returning int.

Q62. How to pass function pointer to another function?

Q63. Callback function example in C.

Q64. Difference: normal vs callback function.

Q65. Array of function pointers.

Q66. Use function pointer to call add(int, int).

Q67. Can function pointers point to static functions?

Q68. What is typedef for function pointer?

Q69. How is function address stored in memory?

Q70. void (*fptr)(); What is this?

Stack

Q71. What is LIFO?

Q72. Implement stack using array.

Q73. What is stack overflow?

Q74. Recursive function and stack.

Q75. Push and pop operations.

Q76. Stack in expression evaluation.

Q77. Call stack frame in recursion.

Q78. Stack using linked list.

100 Logical Reasoning Questions in C Programming

Q79. Check stack underflow.

Q80. Complexity of push/pop?

Heap and Memory Errors

Q81. malloc vs calloc?

Q82. What is segmentation fault?

Q83. Accessing freed memory.

Q84. Double free error.

Q85. Memory leak example.

Q86. Dangling pointer usage.

Q87. Heap vs Stack memory.

Q88. Segfault on NULL pointer dereference.

Q89. Use-after-free vulnerability.

Q90. Tools to detect memory errors.